

# THE MAGIC EXPRESS

## Speeding Up Lists & Python Programming

PLANRS Academy — Grade 4

### The Hook: Sieve vs. Hand-picking

---

#### Imagine This...

Imagine you have a giant basket of mangoes. Some are ripe and yellow (**Alphonso!**), and some are still green.

***Visual representation:** A child picking mangoes one by one vs. using a magic sieve that filters them instantly.*

If you pick them one by one, it takes forever! But what if you had a **Magic Sieve**? You pour the basket in, and **BAM!**—only the ripe ones fall into your tray.

In Python, **List Comprehension** is our Magic Sieve. It's the "Express Route" to making lists faster and smarter!

# What is List Comprehension?

---

## One Line Instead of Many

Usually, we use a 'for-loop' to make a list. That takes many lines. With List Comprehension, we do it in just **ONE line!**

```
The Blueprint:  
[ x for x in list ]
```

It's like taking the **Vande Bharat Express** instead of a slow passenger train. You get the same result, but much faster!

**Visual representation:** Comparison of a long winding road (Loop) and a straight high-speed train track (Comprehension).

## Coding Shortcuts

---

### Old Way vs. New Way

Let's look at how Reyansh creates a list of numbers:

#### OLD WAY (THE LONG ROAD)

```
numbers = [1, 2, 3, 4]
new_list = []
for n in numbers:
    new_list.append(n * 2)

# Takes 4 lines!
```

#### NEW WAY (THE MAGIC EXPRESS)

```
numbers = [1, 2, 3, 4]
new_list = [n * 2 for n in numbers]

# Just 1 line!
```

Both methods do the exact same thing, but the **Magic Express** is cleaner, shorter, and much easier to read.

# The Art of Sorting

---

## Organizing Your World

Sorting means putting things in order. Think of a **Kirana Store** shopkeeper arranging spice packets from the smallest price to the largest.

***Visual representation:** A shopkeeper neatly arranging jars on a shelf.*

There are two primary ways to sort lists in Python:

### **.sort()**

Changes your original list directly and makes it **Permanent**.

### **sorted()**

Keeps the old list completely safe and gives you a brand-new, tidy one!

```
prices = [50, 10, 30]
prices.sort()
print(prices) # Result: [10, 30, 50]
```

## Hands-on Activity

---

### Activity: Gali Cricket Scoreboard

Advait and his friends played cricket. Here are their scores: **[12, 7, 19, 5, 22]**.

Let's find out who the top scorer is using Python code!

#### Ascending (Low to High)

```
scores = [12, 7, 19, 5, 22]
print(sorted(scores))
# Output: [5, 7, 12, 19, 22]
```

#### Descending (High to Low)

To go from top to bottom (like coming down temple stairs), we use the secret word **reverse=True**.

```
print(sorted(scores, reverse=True))
# Output: [22, 19, 12, 7, 5]
```

**The winner scored 22 runs!**

# The One-Line Challenge

---

## Why Use This?

Coding Ninjas use these tools to keep their code organized, just like a Librarian arranges books or a student lines up neatly for the Morning Assembly.

### Quick Quiz:

1. Which is faster: A for-loop or List Comprehension?
2. If we climb temple stairs, is that Ascending or Descending?
3. How do you sort a list from High to Low?

**Awesome job today! You are now a List Comprehension Ninja! 🎉**

# PLANRS Academy



Empowering tomorrow's Coding Ninjas, one line at a time.

© 2026 PLANRS Academy. All rights reserved.

## Practice Arena

Test your skills and become a true Coding Ninja! (No Answers Provided)

### Answer the following questions based on today's lesson:

1. What is the special name given to the "Magic Sieve" in Python that creates lists in just one line?  
.....
2. If a standard for-loop takes about 4 lines of code to double a list, how many lines does the Magic Express route take?  
.....
3. Write down the missing word in the List Comprehension Blueprint:  
`[ x for x in _____ ]`  
.....
4. If you arrange your toy cars from the cheapest price to the most expensive price, are you sorting them in **Ascending** or **Descending** order?  
.....
5. Which sorting tool completely changes your original list permanently: **.sort()** or **sorted()**?  
.....
6. If you want to keep your original list safe and untouched but still want to look at a sorted version of it, which function should you use?  
.....
7. Imagine you are walking down a grand flight of temple stairs. Is this an example of **Ascending** or **Descending** order?  
.....
8. If Reyansh has a list **scores = [10, 5, 20]**, what will Python print when he types **print(sorted(scores))**?  
.....
9. What is the exact keyword phrase we must add inside the **sorted()** function to sort a list from High to Low?  
.....
10. Why do you think a Librarian or a shopkeeper likes to keep things sorted? How does it help them?  
.....
11. If you have a list of words **["Mango", "Apple", "Banana"]** and you use **.sort()**, which fruit will appear first in the list? (Hint: Think about the A-B-C alphabet order!)  
.....



12. Look at this code: `[n * 2 for n in [1, 2, 3]]`. What is this line of code doing to each number inside the list?